

AutoCamTracker V1

AI Development Specification / AI 開發規格書

本文件可直接交給 AI 開發工具使用。目標是用 Python、OpenCV、Tkinter、Ultralytics YOLO Track Mode 建立 V1：支援 MacBook webcam、本機影片、螢幕區域輸入；使用 BoT-SORT 與 Deep OC-SORT 追蹤車輛；提供 before / after 同步畫面、遠景 / 中景 / 特寫 reframe、錄影與評估紀錄。

Language / 語言

Python

UI / 介面

Tkinter

Vision / 影像處理

OpenCV

Detector / 偵測

Ultralytics YOLO

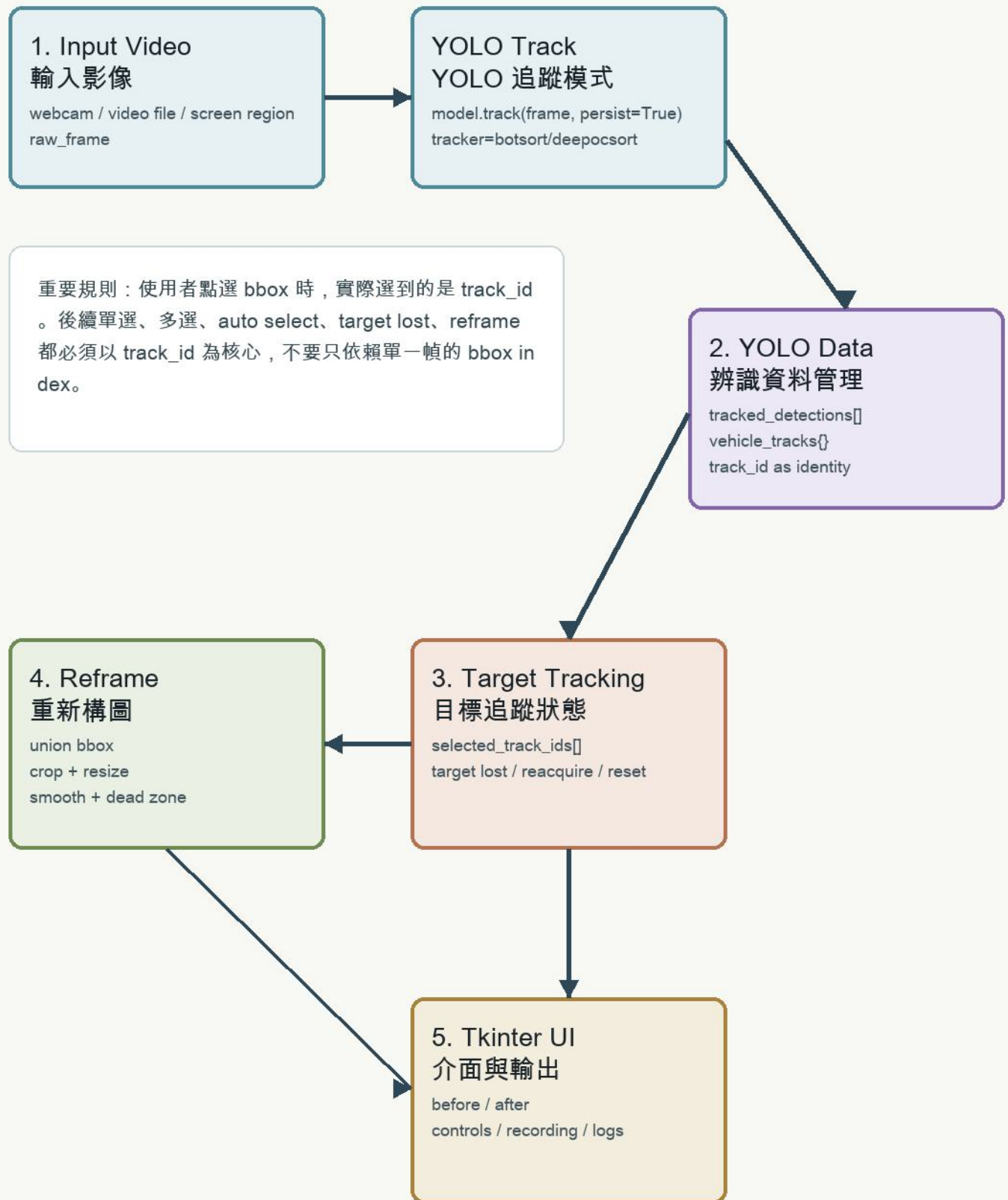
Trackers / 追蹤器

BoT-SORT, Deep OC-SORT

Webcam / 鏡頭

MacBook camera via cv2.VideoCapture(0)

資料流動圖 / Data Flow Diagram



開發總規格 / Overall Requirements

專案目標 / Project Goal

- 建立 Python 桌面應用程式，用於車輛偵測、追蹤、自動 reframe 與 demo recording。
- 支援 MacBook webcam、本機影片檔、螢幕區域擷取三種 input source。
- 使用 Ultralytics YOLO Track Mode，並支援 BoT-SORT 與 Deep OC-SORT tracker。
- UI 使用 Tkinter，需同步顯示 before view 與 after view。
- 追蹤目標可單選或多選，reframe 支援遠景、中景、特寫。

技術條件 / Technical Requirements

- Programming Language / 程式語言：Python。
- UI Framework / 介面框架：Tkinter。
- Computer Vision / 影像處理：OpenCV。
- Detection and Tracking / 偵測與追蹤：Ultralytics YOLO Python API。
- Default Tracker / 預設追蹤器：BoT-SORT，tracker config 使用 botsort.yaml。
- Optional Tracker / 可選追蹤器：Deep OC-SORT，tracker config 使用 deepocsort.yaml。
- MacBook Webcam / MacBook 鏡頭：預設使用 cv2.VideoCapture(0)，並允許調整 camera index。
- Target Identity / 目標身份：必須使用 Ultralytics tracking result 的 track_id 作為車輛身份。

Ultralytics 使用規則 / Ultralytics Usage Rules

```
from ultralytics import YOLO

model = YOLO("yolo26n.pt") # 或 path/to/best.pt
results = model.track(
    frame,
    persist=True,
    tracker="botsort.yaml", # or "deepocsort.yaml"
    conf=0.25,
    iou=0.7,
)
```

- persist=True 必須啟用，讓 tracker 在連續 frame 之間維持 track_id。
- track_id 來源應從 result.bboxes.id 取得；若 result.bboxes.is_track 為 False，該幀不可更新 selected target。
- BoT-SORT 適合一般與移動鏡頭場景，作為 V1 預設。
- Deep OC-SORT 適合擁擠或容易 ID swap 的 moving-camera 場景，作為可切換選項。

1. Input Video + YOLO Detect

1. Input Video + YOLO Detection / 輸入影片與 YOLO 影像辨識

功能與範圍：負責取得影像來源，載入 YOLO 模型，對每一幀執行 `model.track()`，輸出 `raw_frame` 與 `tracked_detections`。

必要功能 / Required Features

- Webcam Input / 網路攝影機輸入：MacBook camera，預設 `index=0`。
- Local Video File Input / 本機影片檔輸入：支援 `mp4`、`mov` 等 OpenCV 可讀格式。
- Screen Region Capture / 螢幕區域擷取：使用者可框選螢幕區域，偵測網路影片或播放器畫面。
- YOLO Model Loading / 載入 YOLO 模型：支援官方模型與自訂 `best.pt`。
- Tracker Selection / 追蹤器選擇：`botsort.yaml` 或 `deepocsort.yaml`。
- Vehicle Classes / 車輛類別：`car`、`truck`、`bus`、`motorcycle`，`person` 可選。

資料介面 / Data Interface

輸入：`source_type`、`camera_index`、`video_path`、`screen_region`、`model_path`、`tracker_name`、`conf`、`iou`。
輸出：`raw_frame`、`tracked_detections[]`、`annotated_detection_frame`。

不負責範圍 / Out of Scope

不負責 `target selection`、`target lost` 決策、`reframe`、`UI layout`、`recording file writing`。

2. YOLO Data / 辨識資料管理

2. YOLO Data / 辨識資料管理

功能與範圍：負責把 `tracked_detections` 整理成穩定的車輛資料，並以 `track_id` 建立 `vehicle_tracks`。

必要功能 / Required Features

- Detection Data Store / 偵測資料儲存：保存目前幀的 `tracked detections`。
- Vehicle Tracks / 車輛軌跡資料：以 `track_id` 為 key 保存最新 `bbox`、`confidence`、`center history`。
- Detection History / 偵測歷史：保留最近 N 幀資料，支援 `reacquire` 與 `debug`。
- Candidate Ranking / 候選排序：提供 `auto select one / multiple` 使用。
- Filtering / 過濾：依 `class`、`confidence threshold` 篩選車輛。

資料介面 / Data Interface

輸入： `tracked_detections[]`、`frame_index`、`timestamp`。

輸出： `current_vehicle_candidates[]`、`vehicle_tracks{}`、`ranked_candidates[]`。

不負責範圍 / Out of Scope

不負責使用者選取、不負責重設追蹤、不負責畫面裁切。

3. Target Tracking / 目標追蹤狀態管

3. Target Tracking / 目標追蹤狀態管理

功能與範圍：負責管理 `selected_track_ids`、單選、多選、auto select、target lost、簡化 reacquire 與 reset。

必要功能 / Required Features

- Manual Select / 手動選取：使用者點 bbox label 後選取對應 `track_id`。
- Single Select / 單選：一次只追蹤一台車。
- Multi Select / 多選：可追蹤多個 `track_id`。
- Auto Select One / 自動選一台：預設選最大 bbox 或最靠近中心車輛。
- Auto Select Multiple / 自動選多台：選前 N 個 ranked candidates。
- Target Lost Handling / 目標遺失處理：超過 `max_lost_frames` 後提醒失敗、清除選取、回到可選狀態。
- Reacquire / 重新捕捉：短暫消失時，用 tracker 與 detection history 嘗試找回。

資料介面 / Data Interface

輸入：`current_vehicle_candidates[]`、`vehicle_tracks{}`、`user_action`、`tracking_config`。

輸出：`selected_targets[]`、`selected_track_ids[]`、`tracking_status`、`lost_alert`。

不負責範圍 / Out of Scope

不負責 YOLO 推論、不負責 OpenCV crop、不負責 Tkinter widget layout。

4. Reframe / Digital Zoom /

4. Reframe / Digital Zoom / Output Frame / 重新構圖、數位變焦與輸出畫面

功能與範圍：負責根據 selected targets 產生 after view：crop + resize、smooth、dead zone、遠景 / 中景 / 特寫。

必要功能 / Required Features

- Single Target Framing / 單目標構圖：用 selected track 的 latest bbox 計算 crop center。
- Multi-target Group Framing / 多目標群組構圖：合併所有 selected bbox 成 union bbox。
- Framing Presets / 構圖預設：Wide Shot 遠景、Medium Shot 中景、Close-up Shot 特寫。
- Smooth Movement / 平滑移動：crop window 逐步靠近目標中心。
- Dead Zone / 死區：小幅偏移不移動 crop，減少抖動。
- Boundary Clamp / 邊界限制：crop window 不可超出 frame。

資料介面 / Data Interface

輸入：raw_frame、selected_targets[]、framing_config。

輸出：tracking_output_frame、before_after_comparison_frame、framing_status。

不負責範圍 / Out of Scope

不負責偵測、不負責選取、不負責錄影寫檔。

5. Tkinter UI + Recording +

5. Tkinter UI + Recording + Debug Log / Tkinter 介面、錄影與除錯紀錄

功能與範圍：負責整合各區塊、顯示 before / after、提供按鈕、錄影與輸出 evaluation log。

必要功能 / Required Features

- Before View / 追蹤前畫面：顯示 raw 或 detection frame、bbox、label、track_id、confidence。
- After View / 追蹤後畫面：顯示 reframe 後的 output frame。
- Input Controls / 輸入控制：webcam、video file、screen region。
- Tracker Controls / 追蹤器控制：BoT-SORT / Deep OC-SORT 切換。
- Selection Controls / 選取控制：single、multi、clear、auto one、auto multiple。
- Framing Controls / 構圖控制：遠景、中景、特寫。
- Status Labels / 狀態標籤：FPS、tracker、selected ids、tracking status、lost count。
- Recording / 錄影：raw、detection、tracking output、before/after comparison。
- Debug Log / 除錯紀錄：FPS、track_id、confidence、crop position、lost events、framing mode。

資料介面 / Data Interface

輸入：raw_frame、tracked_detections、selected_targets、tracking_output_frame、status objects、user actions。
輸出：Tkinter display、video files、debug_log.csv 或 debug_log.json。

不負責範圍 / Out of Scope

UI 不應放置核心 tracking 演算法；只負責呼叫 Target Tracking 模組並呈現結果。

流程與驗收 / Flow and Acceptance

每一幀處理流程 / Per-frame Pipeline

```
raw_frame = input.read()
tracked_detections = yolo.track(raw_frame, tracker, persist=True)
vehicle_tracks = detection_store.update(tracked_detections)
selected_targets = target_tracker.update(vehicle_tracks, user_actions)
tracking_output_frame = reframer.render(raw_frame, selected_targets)
ui.update(before_frame, tracking_output_frame, status)
recorder.write_if_enabled(...)
logger.write_frame_metrics(...)
```

Target Lost 流程 / Target Lost Flow

- 若 selected track_id 沒有出現在目前 frame , lost_frame_count + 1。
- 若 lost_frame_count 未超過 max_lost_frames , 維持上一個 crop 或嘗試 reacquire。
- 若超過 max_lost_frames , tracking_status = failed。
- UI 顯示追蹤失敗提醒, 清除 selected_track_ids , 回到可重新選取狀態。

V1 驗收條件 / Acceptance Criteria

- MacBook webcam 可透過 OpenCV index 0 啟動並顯示 live frame。
- 本機影片檔可載入、播放、暫停、停止。
- 螢幕區域擷取可作為 input source。
- YOLO tracking 能在 before view 顯示 bbox、track_id、confidence。
- BoT-SORT 與 Deep OC-SORT 可在 UI 或 config 切換。
- 使用者可點選 bbox 追蹤單台或多台車。
- Auto select one / multiple 可根據 ranked candidates 選目標。
- After view 可根據 selected targets 自動 reframe。
- 遠景 / 中景 / 特寫三種構圖模式可切換。
- Target lost 時 UI 會提醒、重置選取, 並回到可重新選取狀態。
- 可輸出 raw、detection、tracking output、before/after comparison recording。
- 可輸出 debug / evaluation log。

建議檔案結構 / Suggested Files

```
video_detector.py    # Input Video + YOLO Detection
detection_store.py   # YOLO Data / track_id data store
target_tracker.py    # Target Tracking state management
reframer.py          # Reframe / Digital Zoom / Output Frame
app.py               # Tkinter UI + Recording + Debug Log
config.py            # model path, tracker, thresholds, UI sizes
```